2/10/24

Exp - 12 a) Implementation of echo client server.

Aim:

To implement echo client server using TCP/UDP sockets.

Algorithm:

i) TCP:

echo server and client:

- Start the TCP echo server and set the server port to the given port or use 5000 by default.
- Create a server socket and listen on the specified port.
- Wait for the client connection and create input and output streams for communicating with client
- Read a message sent by the client and display the received message and send it back to the client with "echo:" added.
- If the received message is bye - close connection otherwise continue receiving messages.

ii) UDP:

echo server and client

- Start the UDP server and listen on port 6000
- Start the UDP client and create a datagram packet.
- Read the message from the client and convert it into a datagram packet and send it to the server.
- The server receives the packet and extracts the message. The server then adds "Echo:" to the packet and send it send the response.
- The client receives the echoed response. If the client sends bye, stop communication

Code : TCP

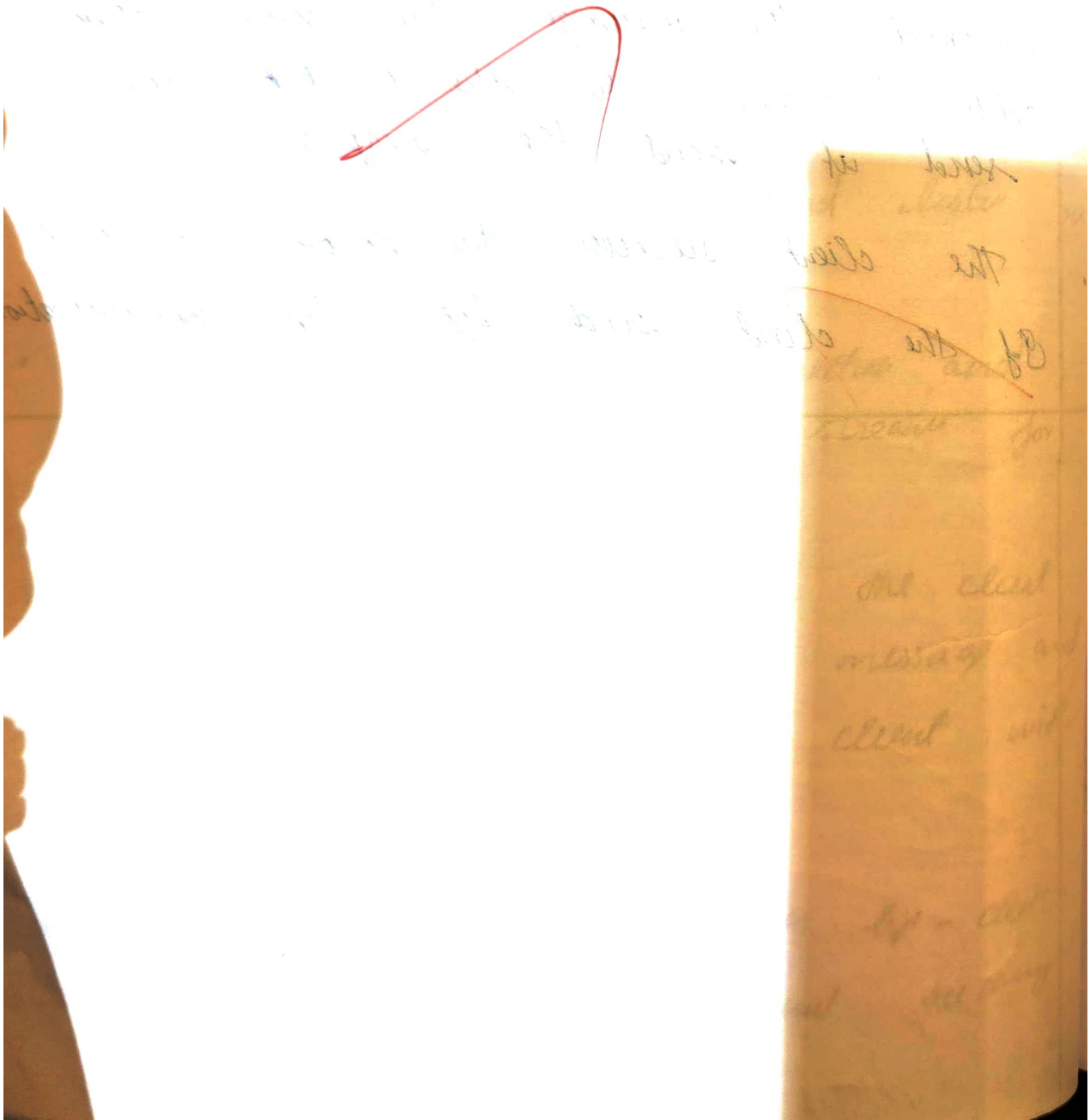
```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

msg = input("Enter message: ")
client.sendto(msg.encode(), ('localhost', 5001))

data, addr = client.recvfrom(1024)
print("Echo from server:", data.decode())

client.close()
```



Output:

```
Enter message: Hello Server
Echo from server: Hello Server
```

Code : UDP

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(('127.0.0.1', 12345))
s.listen(1)

print("TCP Server is waiting for client...")
conn, addr = s.accept()
print("Client connected:", addr)

data = conn.recv(1024).decode()
print("Client message:", data)

conn.send(data.encode())
conn.close()
s.close()
```





Output:

```
UDP Server is ready...
Received: Hello Computer Networks
```

Result:

The implementation of echo client server using TCP and UDP has been completed successfully.

2/10/26 Exp : 12 b) Implementation of chat program

Aim:

To implement a chat program using socket programming that enables communication between a client and a server through a network connection.

Algorithm:

- Start the chat server on port 5000 and wait for client connections.
- Ask the client to enter a name and add the client to the connected client list
- Start the client and establish a socket connection with the server.
- The client sends messages entered by the user to the server.
- The server receives each message and broadcasts it to all other connected clients.
- The client concurrently listens for messages from the server using a special listener thread.

If a client sends "/quit", terminate that clients connection.

Remove the disconnected client and notify the remaining clients.

Continue accepting and managing new clients until the server is stopped.

Code:

```
print("What does the Visualize button do?"import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(('localhost', 5000))
s.listen(1)
print("TCP Chat Server Started...")
conn, addr = s.accept()
print("Client connected")
while True:
    msg = conn.recv(1024).decode()
    print("Client:", msg)
    if msg.lower() == "exit":
        break
    reply = input("Server: ")
    conn.send(reply.encode())
    if reply.lower() == "exit":
        break
conn.close()
s.close(
```

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.bind(('localhost', 5001))
print("UDP Chat Server Started...")
while True:
    msg, addr = s.recvfrom(1024)
    msg = msg.decode()
    print("Client:", msg)
    if msg.lower() == "exit":
        break
    reply = input("Server: ")
    s.sendto(reply.encode(), addr)
    if reply.lower() == "exit":
        break
s.close()
```

Output :

```
UDP Chat Server Started...
Client: Hello
Server: Hi
Client: How are you?
Server: I am fine
Client: exit
```

```
TCP Chat Server Started...
Client connected
Client: Hello
Server: Hi
Client: How are you?
Server: I am fine
Client: exit
```

Result :

The implementation of chat program is implemented successfully.